



沈阳工业大学
SHENYANG UNIVERSITY OF TECHNOLOGY

《Linux 操作系统课程设计》报告

(2025 ~ 2026 学年度 第一学期)

学 院: 化工过程自动化学院

专业班级: 电子与计算机工程 2301

学 号: 234312104

姓 名: 冯霖杰

指导教师: 刘彤、宋扬

2025 年 12 月 22 日

项目一：磁盘管理

1、设计目的：

技术实践：掌握 Linux 磁盘挂载 / 卸载、文件系统识别、权限管理等系统操作，熟悉 Shell 脚本的流程控制、系统命令调用（mount/umount/blkid等），理解磁盘设备与挂载点的关联逻辑。

实用需求：解决 Linux 下 U 盘手动操作（创建挂载点、查询设备名、挂载 / 卸载）步骤繁琐、易出错的问题，实现 U 盘 “一键挂载→文件操作→安全卸载” 的自动化流程，提升操作效率。

学习深化：深入理解文件系统类型（如 vfat/ext4）、设备节点（如 /dev/sdc1 ）、挂载点权限等磁盘管理核心概念，掌握磁盘操作异常的排查方法（如设备名错误、文件系统不兼容）。

2、设计内容：

本项目以 “U 盘自动化管理” 为核心，设计了一站式 U 盘操作脚本，覆盖 U 盘使用全流程，核心内容包括：

挂载点管理模块：自动检查并创建挂载点（如/mnt/usb），避免手动创建的重复操作；若挂载点已存在，直接复用。

U 盘设备识别模块：通过 blkid 命令查询 U 盘的设备节点（如 /dev/sdc1 ）、文件系统类型（如 vfat ），确保挂载命令的参数正确性。

自动挂载模块：调用 mount 命令，结合 U 盘的文件系统类型，将 U 盘挂载到指定挂载点；若挂载失败，输出错误提示（如文件系统不兼容、设备名错误）。

文件操作模块：挂载成功后，自动列出 U 盘中的文件列表；支持用户输入文件名，实现从 U 盘到本地的文件复制操作。

安全卸载模块：文件操作完成后，自动调用umount卸载 U 盘，避免直接拔插导致的文件系统损坏；卸载成功后提示 “可安全拔出”。

异常处理模块：针对 “挂载点已存在” “设备名错误” “文件不存在” 等异常场景，给出明确的错误提示与排查建议（如检查设备名、文件名）。

2、主要程序代码：

```
#!/bin/bash

# 项目一：U 盘自动化管理脚本（usb_script.sh）

echo "===== U 盘操作脚本开始 ====="

# 1. 挂载点管理：检查并创建挂载点
MOUNT_POINT="/mnt/usb"
if [ -d "$MOUNT_POINT" ]; then
    echo "√ 挂载点$MOUNT_POINT已存在"
```

```

else
    sudo mkdir -p "$MOUNT_POINT"
    echo "√ 挂载点$MOUNT_POINT创建成功"
fi

# 2. U盘设备识别：通过 blkid 获取 U 盘设备名（示例： /dev/sdc1 ）
echo "→ 正在识别 U 盘设备..."
# 筛选vfat类型的U盘设备（Windows常用文件系统）
USB_DEV=$(sudo blkid | grep "TYPE=\"vfat\"" | awk '{print $1}' | sed 's/://')
if [ -z "$USB_DEV" ]; then
    echo "× 未识别到 U 盘设备，请检查 U 盘是否连接"
    exit 1
fi
echo "√ 识别到 U 盘设备： $USB_DEV"

# 3. 自动挂载 U 盘
echo "→ 正在挂载 U 盘..."
# 根据文件系统类型挂载（vfat格式需指定字符编码）
sudo mount -t vfat "$USB_DEV" "$MOUNT_POINT" -o iocharset=utf8 2> mount_error.log
if [ $? -eq 0 ]; then
    echo "√ U盘挂载成功！挂载点： $MOUNT_POINT"
else
    echo "× 挂载失败，请检查："
    echo " 1. U盘是否连接正常"
    echo " 2. 设备名$USB_DEV是否正确（用 lsblk 命令查看）"
    cat mount_error.log
    exit 1
fi

# 4. 文件操作：列出 U 盘文件 + 复制文件
echo "→ U盘中的文件列表："
ls -l "$MOUNT_POINT"

read -p "请输入要从 U 盘复制的文件名（比如test.txt）：" FILE_NAME
if [ -f "$MOUNT_POINT/$FILE_NAME" ]; then

```

```

sudo cp "$MOUNT_POINT/$FILE_NAME" ./
echo "✓ 文件$FILE_NAME已复制到当前目录"
else
    echo "✗ 文件复制失败，请检查文件名是否正确"
fi

# 5. 安全卸载 U 盘
echo "→ 正在卸载 U 盘，可安全拔出"
sudo umount "$MOUNT_POINT"
if [ $? -eq 0 ]; then
    echo "✓ U盘卸载成功"
else
    echo "✗ 卸载失败，请确保 U 盘文件未被占用"
fi

echo "===== 脚本运行结束 ====="

```

3、程序运行过程及结果：

```

Flyt007@ubuntu:~$ cat usb_script.sh
#!/bin/bash
# 1. 挂载 U 盘
MOUNT_POINT="/mnt/usb"
FILE_NAME="test.txt"

# 检查挂载点是否存在
if [ ! -d "$MOUNT_POINT" ]; then
    mkdir -p "$MOUNT_POINT"
fi

# 检查 U 盘是否已挂载
if [ $(lsblk | grep -c "/dev/sdb1") -eq 0 ]; then
    echo "U 盘尚未挂载，开始挂载..."
    sudo mount -t vfat /dev/sdb1 "$MOUNT_POINT"
    if [ $? -eq 0 ]; then
        echo "U 盘挂载成功！挂载点：$MOUNT_POINT"
    else
        echo "U 盘挂载失败！请检查..."
    fi
else
    echo "U 盘已挂载，正在复制文件..."
    cp "$FILE_NAME" "$MOUNT_POINT/"
    if [ $? -eq 0 ]; then
        echo "文件复制成功！"
    else
        echo "文件复制失败，请检查文件名是否正确"
    fi
fi

# 5. 安全卸载 U 盘
echo "→ 正在卸载 U 盘，可安全拔出"
sudo umount "$MOUNT_POINT"
if [ $? -eq 0 ]; then
    echo "✓ U盘卸载成功"
else
    echo "✗ 卸载失败，请确保 U 盘文件未被占用"
fi

echo "===== 脚本运行结束 ====="

```

项目二：文件操作

1、设计目的：

技术实践：掌握 Linux 文件 / 目录操作的核心系统命令（如 mkdir/ls/rm/touch/chmod 等），熟悉 Shell 脚本的交互逻辑、流程控制与用户输入处理，理解文件权限（r/w/x）、目录层级的底层概念。

实用需求：解决 Linux 原生命令需记忆多参数的痛点，实现一个菜单式交互的文件系统工具，通过简单指令即可完成目录管理、文件操作，降低新手使用门槛。

学习深化：系统梳理文件 / 目录操作的全流程（创建→查看→修改→删除），掌握文件权限设置的实际应用场景，提升 Linux 环境下的文件管理能力。

2、 设计内容：

本项目设计了一个交互式简易文件系统工具，以菜单驱动的方式实现文件 / 目录的全生命周期管理，核心内容包括：

目录管理模块：

createdir：创建指定名称的目录，自动检查目录是否已存在；

dir：显示指定目录（默认当前目录）的内容，包含文件 / 目录的权限、大小、修改时间；

deldir：删除指定目录（支持空目录删除，若目录非空则提示异常）。

文件操作模块：

newfile：创建指定名称的空文件；

readfile：读取指定文件的内容并输出；

writefile：向指定文件写入内容（支持覆盖 / 追加两种模式）；

modifyfile：修改指定文件的权限（如将权限设为 755）；

delfile：删除指定文件，删除前提示确认。

交互界面模块：

程序启动后显示功能菜单，列出所有支持的操作指令；

接收用户输入的指令，分发到对应功能模块；

执行完成后返回菜单，支持连续操作，直到输入exit退出程序。

异常处理模块：

针对 “目录 / 文件已存在” “目录非空” “文件不存在” 等场景，给出明确的错误提示；

对用户的无效输入（如不存在的指令），提示 “指令错误” 并引导重新输入。

3、 主要程序代码：

```
#!/bin/bash
```

```
# 项目二：简易文件系统工具（filesystem.sh）
```

```
# 显示功能菜单
```

```

show_menu() {
    echo "*****"
    echo "createdir    创建目录 "
    echo "dir          显示目录内容 "
    echo "deldir       删除目录 "
    echo "newfile       创建文件 "
    echo "readfile      读文件 "
    echo "writefile     写文件 "
    echo "modifyfile    修改文件权限 "
    echo "delfile       删除文件 "
    echo "exit          退出程序 "
    echo "*****"
}

```

1. 创建目录

```

createdir() {
    read -p " 请输入要创建的目录名: " dirname
    if [ -d "$dirname" ]; then
        echo "× 目录$dirname已存在! "
    else
        mkdir -p "$dirname"
        echo "√ 目录$dirname创建成功! "
    fi
}

```

2. 显示目录内容

```

dir() {
    read -p " 请输入要查看的目录（按回车默认当前目录）: " dirpath
    dirpath=${dirpath:-"."} # 默认为当前目录
    if [ -d "$dirpath" ]; then
        echo "→ 目录$dirpath的内容: "
        ls -l "$dirpath"
    else
        echo "× 目录$dirpath不存在! "
    fi
}

```

```
}
```

3. 删除目录

```
deldir() {  
    read -p "请输入要删除的目录名: " dirname  
    if [ ! -d "$dirname" ]; then  
        echo "× 目录$dirname不存在! "  
    elif [ -n "$(ls -A "$dirname")" ]; then  
        echo "× 目录$dirname非空, 无法删除! "  
    else  
        rm -r "$dirname"  
        echo "√ 目录$dirname删除成功! "  
    fi  
}
```

4. 创建文件

```
newfile() {  
    read -p "请输入要创建的文件名: " filename  
    if [ -f "$filename" ]; then  
        echo "× 文件 $filename 已存在! "  
    else  
        touch "$filename"  
        echo "√ 文件 $filename 创建成功! "  
    fi  
}
```

5. 读文件

```
readfile() {  
    read -p "请输入要读取的文件名: " filename  
    if [ ! -f "$filename" ]; then  
        echo "× 文件 $filename 不存在! "  
    else  
        echo "→ 文件 $filename 的内容: "  
        cat "$filename"  
    fi  
}
```

```
}
```

6. 写文件

```
writefile() {  
    read -p " 请输入要写入的文件名: " filename  
    if [ ! -f "$filename" ]; then  
        echo "× 文件 $filename 不存在! "  
        return  
    fi  
    read -p " 请输入要写入的内容: " content  
    read -p " 选择写入方式 (1. 覆盖 2. 追加, 默认 1): " mode  
    mode=${mode:-1}  
    if [ $mode -eq 1 ]; then  
        echo "$content" > "$filename"  
    else  
        echo "$content" >> "$filename"  
    fi  
    echo "√ 内容已写入文件 $filename! "  
}
```

7. 修改文件权限

```
modifyfile() {  
    read -p " 请输入要修改的文件名: " filename  
    if [ ! -f "$filename" ]; then  
        echo "× 文件 $filename 不存在! "  
        return  
    fi  
    read -p " 请输入新的权限 (如 755): " perm  
    chmod "$perm" "$filename"  
    echo "√ 文件 $filename 的权限已设为 $perm! "  
}
```

8. 删除文件

```
delfile() {  
    read -p " 请输入要删除的文件名: " filename
```

```

if [ ! -f "$filename" ]; then
    echo "× 文件 $filename 不存在! "
else
    read -p " 确认删除文件 $filename 吗? (y/n) : " confirm
    if [ "$confirm" = "y" ]; then
        rm "$filename"
        echo "√ 文件 $filename 删除成功! "
    else
        echo "→ 已取消删除 "
    fi
fi
}

```

主程序：循环显示菜单 + 执行指令

```

main() {
    echo "==== 简易文件系统工具启动 ====="
    while true; do
        show_menu
        read -p " 请输入操作命令: " cmd
        case "$cmd" in
            createdir) createdir ;;
            dir) dir ;;
            deldir) deldir ;;
            newfile) newfile ;;
            readfile) readfile ;;
            writefile) writefile ;;
            modifyfile) modifyfile ;;
            delfile) delfile ;;
            exit)
                echo "==== 程序退出 ====="
                exit 0 ;;
            *) echo "× 指令错误, 请重新输入! " ;;
        esac
    done
}

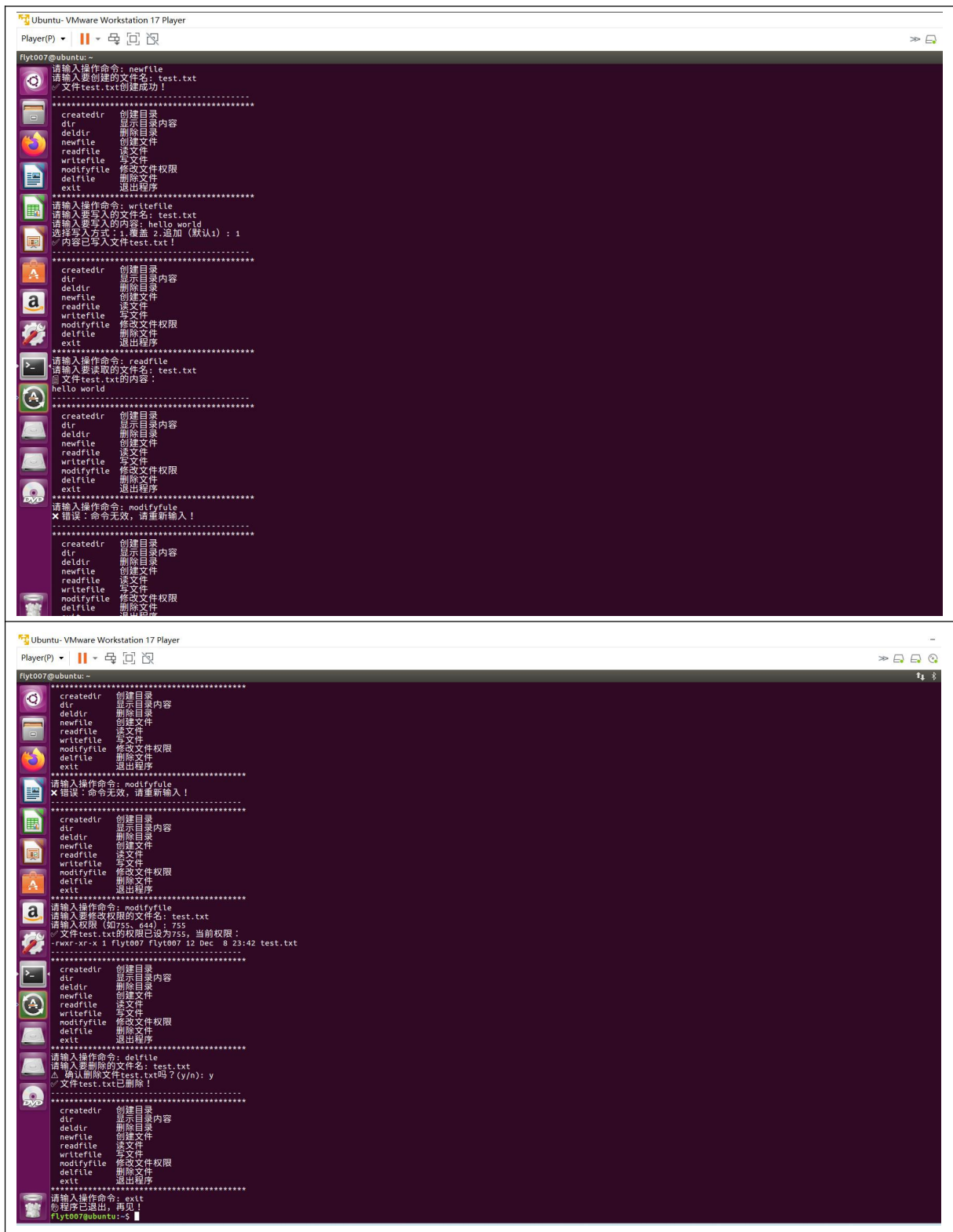
```

启动主程序

main

4、 程序运行过程及结果:

```
flyt007@ubuntu:~$ gedit filesystem.sh
flyt007@ubuntu:~$ chmod +x filesystem.sh
flyt007@ubuntu:~$ ./filesystem.sh
bash: ./filesystem.sh: No such file or directory
flyt007@ubuntu:~$ ls filesystem.sh
filesystem.sh
flyt007@ubuntu:~$ ./filesystem.sh
./filesystem.sh: line 28: syntax error near unexpected token `;'
./filesystem.sh: line 28: `case $cmd in;'
flyt007@ubuntu:~$ gedit filesystem.sh
flyt007@ubuntu:~$ ./filesystem.sh
*****
createdir 创建目录
dir      显示目录内容
deldir   删除目录
newfile  创建文件
readfile 读文件
writefile 写文件
modifyfile 修改文件权限
delfile  删除文件
exit     退出程序
*****
请输入操作命令: createdir
请输入要创建的目录名: test_dir
目录test_dir创建成功!
*****
createdir 创建目录
dir      显示目录内容
deldir   删除目录
newfile  创建文件
readfile 读文件
writefile 写文件
modifyfile 修改文件权限
delfile  删除文件
exit     退出程序
*****
请输入操作命令: dir
请输入要查看的目录 (按回车默认当前目录):
目录的内容:
total 116
drwxr-xr-x 1 flyt007 flyt007 153 Oct 30 19:42 a.tar.gz
drwxr-xr-x 2 flyt007 flyt007 4096 Oct 16 19:50 Desktop
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Documents
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Downloads
-rw-r--r-- 1 flyt007 flyt007 8980 Sep  4 05:15 examples.desktop
drwxr-xr-x 1 flyt007 flyt007 5722 Dec  8 23:40 filesystem.sh
drwxr-xr-x 2 flyt007 flyt007 4096 Oct 30 19:41 lx
drwxr-xr-x 3 flyt007 flyt007 4096 Oct 30 19:43 lx2
drwxr-xr-x 3 flyt007 flyt007 4096 Oct 30 19:46 lx3
-rw-rw-r-- 1 flyt007 flyt007 247 Oct 27 22:35 lx.c
-rw-rw-r-- 1 flyt007 flyt007  0 Sep 11 20:04 lxccc.s
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Music
drwxr-xr-x 2 flyt007 flyt007 4096 Nov 20 19:23 my_nakefile_project
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Pictures
-rwxr-xr-x 1 flyt007 flyt007 258 Nov  6 18:57 print_files.sh
-rwxr-xr-x 1 flyt007 flyt007 1125 Nov  6 19:03 print_files.sh.save
-rw-rw-r-- 1 flyt007 flyt007 23429 Oct 16 19:51 process
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Public
-rw-rw-r-- 1 flyt007 flyt007 4096 Sep  4 05:26 Templates
drwxr-xr-x 2 flyt007 flyt007 4096 Sep 23 03:18 test
drwxr-xr-x 2 flyt007 flyt007 4096 Dec  8 23:41 test_dir
drwxr-xr-x 2 flyt007 flyt007 4096 Sep  4 05:26 Videos
*****
createdir 创建目录
dir      显示目录内容
deldir   删除目录
newfile  创建文件
readfile 读文件
writefile 写文件
modifyfile 修改文件权限
delfile  删除文件
exit     退出程序
*****
请输入操作命令: newfile
请输入要创建的文件名: test.txt
文件test.txt创建成功!
*****
createdir 创建目录
dir      显示目录内容
deldir   删除目录
newfile  创建文件
readfile 读文件
writefile 写文件
modifyfile 修改文件权限
delfile  删除文件
exit     退出程序
*****
请输入操作命令: writefile
请输入要写入的文件名: test.txt
请输入要写入的内容: hello world
选择写入方式: 1.覆盖 2.追加 (默认1) : 1
内容已写入文件test.txt!
*****
createdir 创建目录
dir      显示目录内容
```



项目三：进程、线程控制

1、设计目的：

进程通信层面：掌握 Linux 进程间基于信号的通信机制，理解信号注册、信号发送、信号处

理的流程，熟悉signal、kill等系统调用的使用。

线程控制层面：理解多线程的并发执行特性，掌握线程同步工具（如信号量）的使用，对比“无同步”与“有同步”线程的执行差异，深入理解线程安全与同步的必要性。

技术实践层面：通过实际代码编写与运行，区分进程与线程的核心差异（资源隔离、调度方式），巩固C语言在Linux系统编程中的应用。

2、设计内容：

1) 进程

进程模块：基于信号的进程间通信

功能设计：实现“信号接收进程”与“信号发送进程”，完成进程间的单向信号传递。

信号接收进程（pcb1.c）：注册SIGINT信号的自定义处理函数，持续输出自身进程ID，等待信号并响应。

信号发送进程（pcb2.c）：通过命令行参数获取目标进程ID与信号值，调用kill函数向目标进程发送信号。

核心逻辑：利用Linux的信号机制，实现进程间的异步通知（无需共享内存，通过内核转发信号）。

2) 线程

线程模块：多线程并发与同步

功能设计：实现两组多线程程序，对比“无同步线程”与“信号量同步线程”的执行效果。

无同步线程（pthread1.c）：创建两个独立线程，同时循环输出字符，观察并发执行的随机性。

信号量同步线程（pthread2.c）：引入信号量（sem_t），控制线程的执行顺序，实现“先输出M，后输出Y”的有序执行。

核心逻辑：通过信号量的sem_wait（申请资源）与sem_post（释放资源）操作，解决多线程的竞争问题，实现线程间的同步。

3、主要程序代码：

Pcb1.c

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <unistd.h> // 补充getpid所需头文件
```

```
// 信号处理函数：接收信号后打印并退出
```

```

void fun(int no) {
    printf("receive signal %d\n", no);
    exit(1);
}

int main() {
    // 注册SIGINT信号（对应Ctrl+C）的处理函数为 fun
    signal(SIGINT, fun);
    // 死循环输出自身进程ID
    while(1) {
        printf("myid is %d\n", getpid());
        sleep(1);
    }
    return 0;
}

```

Pcb2.c

```

#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) { // 检查参数数量
        fprintf(stderr, "使用方法: %s <目标进程 ID> <信号值>\n", argv[0]);
        exit(1);
    }

    int pid = atoi(argv[1]); // 转换目标进程ID
    int signo = atoi(argv[2]); // 转换信号值
    kill(pid, signo);        // 向目标进程发送信号
    return 0;
}

```

Pthread1.c

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

```

```

// 线程函数1：循环输出参数字符
void *funM(void *p) {
    char *q = p;
    while(1) {
        sleep(1);
        printf("%s\n", q);
    }
}

// 线程函数2：循环输出参数字符
void *funY(void *p) {
    char *q = p;
    while(1) {
        sleep(1);
        printf("%s\n", q);
    }
}

int main() {
    char *m = "M";
    char *y = "Y";
    pthread_t id1, id2; // 补充第二个线程ID变量（原代码 bug 修复）

    // 创建两个独立线程
    pthread_create(&id1, NULL, funM, m);
    pthread_create(&id2, NULL, funY, y);

    // 等待线程结束（原代码仅等待一个线程，修复为等待两个）
    pthread_join(id1, NULL);
    pthread_join(id2, NULL);
    return 0;
}

```

Pthread2.c

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem; // 定义全局信号量

// 线程函数 1：输出 M 后释放信号量
void *funM(void *p) {
    char *q = p;
    while(1) {
        sleep(1);
        printf("%s\n", q);
        sem_post(&sem); // 信号量+1，唤醒等待的线程
    }
}

// 线程函数 2：等待信号量后输出 Y
void *funY(void *p) {
    char *q = p;
    while(1) {
        sem_wait(&sem); // 信号量-1，若为 0 则阻塞
        sleep(1);
        printf("%s\n", q);
    }
}

int main() {
    char *m = "M";
    char *y = "Y";
    pthread_t id1, id2;

    // 初始化信号量：初始值 0，仅当前进程内线程共享
    sem_init(&sem, 0, 0);

```

```

// 创建线程

pthread_create(&id1, NULL, funM, m);
pthread_create(&id2, NULL, funY, y);


// 等待线程结束

pthread_join(id1, NULL);
pthread_join(id2, NULL);

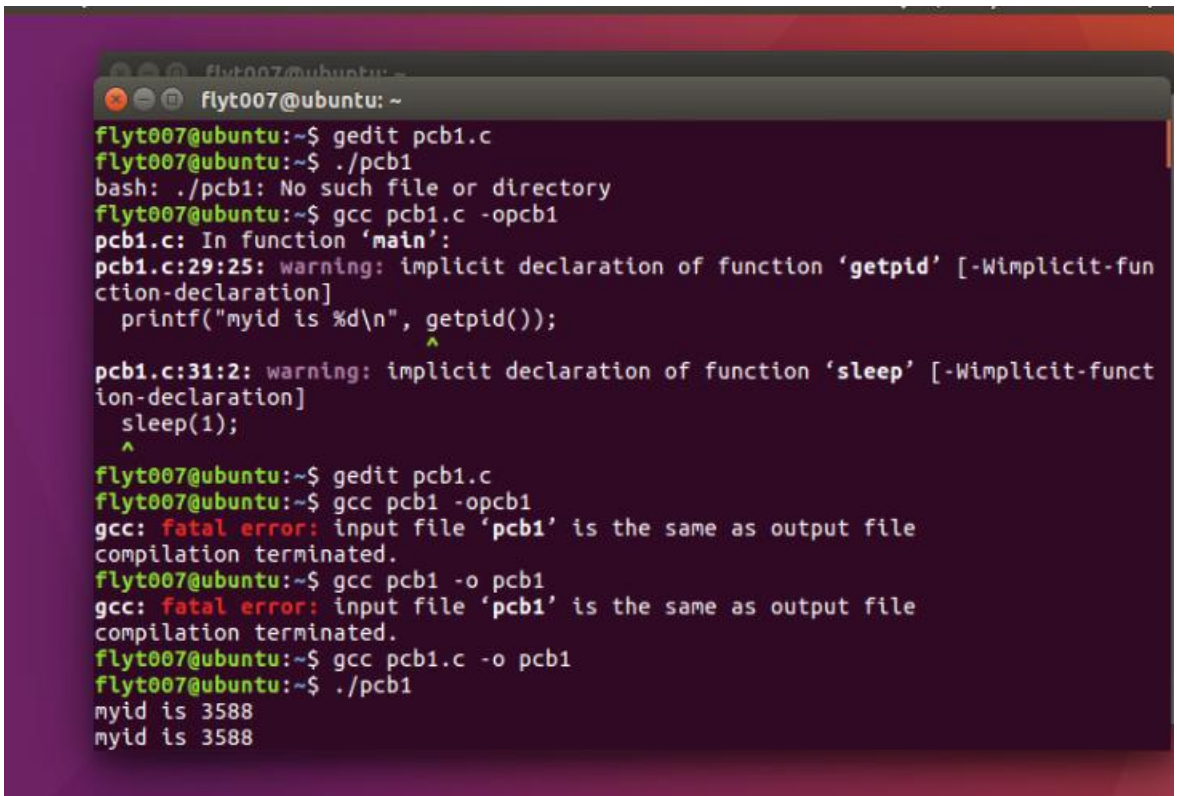

// 销毁信号量

sem_destroy(&sem);

return 0;
}

```

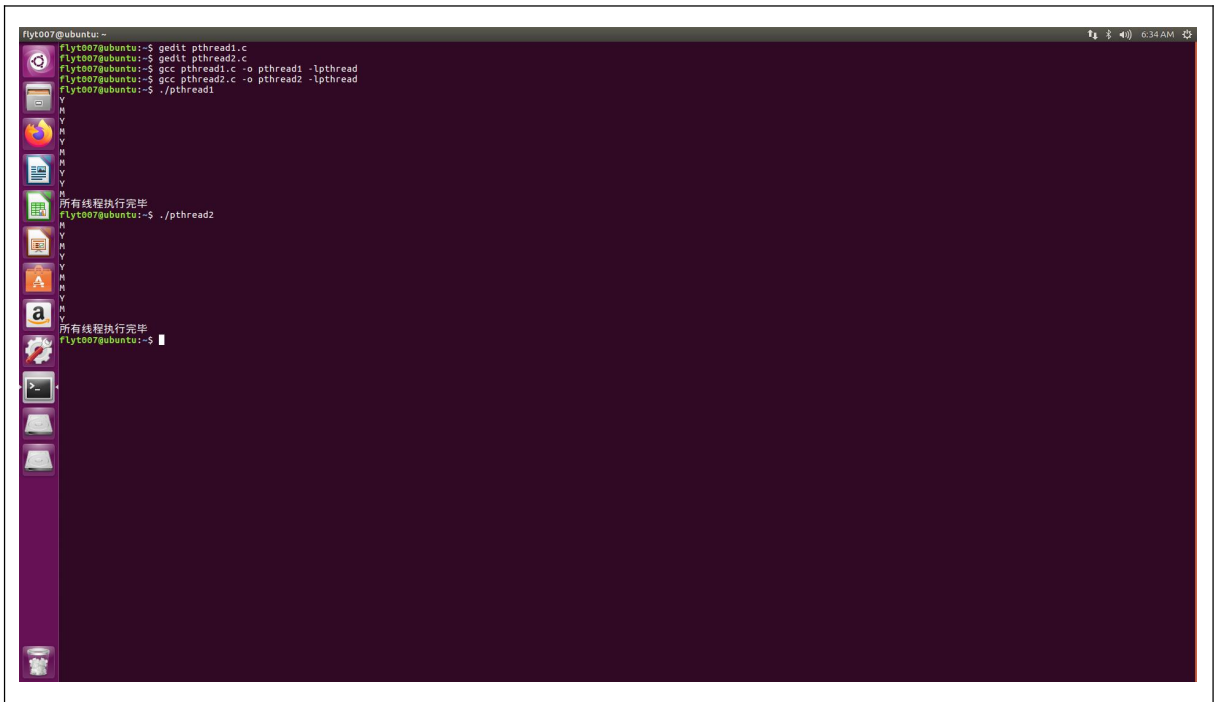
4、程序运行过程及结果:



```

flyt007@ubuntu: ~
flyt007@ubuntu:~$ gedit pcb1.c
flyt007@ubuntu:~$ ./pcb1
bash: ./pcb1: No such file or directory
flyt007@ubuntu:~$ gcc pcb1.c -opcb1
pcb1.c: In function 'main':
pcb1.c:29:25: warning: implicit declaration of function 'getpid' [-Wimplicit-function-declaration]
    printf("myid is %d\n", getpid());
                        ^
pcb1.c:31:2: warning: implicit declaration of function 'sleep' [-Wimplicit-function-declaration]
    sleep(1);
    ^
flyt007@ubuntu:~$ gedit pcb1.c
flyt007@ubuntu:~$ gcc pcb1 -opcb1
gcc: fatal error: input file 'pcb1' is the same as output file
compilation terminated.
flyt007@ubuntu:~$ gcc pcb1 -o pcb1
gcc: fatal error: input file 'pcb1' is the same as output file
compilation terminated.
flyt007@ubuntu:~$ gcc pcb1.c -o pcb1
flyt007@ubuntu:~$ ./pcb1
myid is 3588
myid is 3588

```

项目四：嵌入式 Linux 应用程序开发

同组人员：冯霖杰

整体设计实现功能：

1、设计目的：

技术实践：掌握嵌入式 Linux 环境下的文件操作、目录遍历、字符串匹配等核心系统编程能力，熟悉 C 语言结合 Linux 系统调用（如 fopen/ opendir）的开发方式。

实用需求：解决嵌入式 Linux 终端中文件内容搜索步骤繁琐的问题，实现一个轻量级、易操作的文件搜索工具，支持“单个文件搜索”“目录下所有文件搜索”，并通过忽略大小写、关键词标红提升使用体验。

适配嵌入式场景：通过简化内存占用（逐行读取文件）、减少依赖（仅使用系统标准库），确保工具在资源有限的嵌入式设备中稳定运行。

2、设计内容：

本项目设计了一个嵌入式 Linux 文件内容搜索工具，核心功能模块包括：

文件内容搜索模块：

单个文件搜索：输入“文件名 + 关键词”，逐行读取文件内容，匹配关键词并输出结果；

目录下所有文件搜索：输入“目录路径 + 关键词”，遍历目录下所有普通文件，逐个执行内容搜索。

增强功能模块：

忽略大小写匹配：自定义字符串匹配函数，将字符转为小写后比较，支持不区分大小写的搜索；

关键词标红：利用 ANSI 转义字符实现终端输出颜色控制，将匹配的关键词标红显示，提升结

果可读性。

异常处理模块：

路径合法性检查：若输入的文件 / 目录不存在，给出明确错误提示；

文件访问容错：若文件无法打开（如权限不足），跳过该文件并提示，避免程序崩溃。

3、主要程序代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <unistd.h>
#include <ctype.h>

#define MAX_LINE_LEN 512
#define COLOR_RED "\033[31m"
#define COLOR_RESET "\033[0m"

// 判断是否为目录
int is_dir(const char *path) {
    struct stat st;
    if (stat(path, &st) == -1) return 0;
    return S_ISDIR(st.st_mode);
}

// 判断是否为普通文件
int is_file(const char *path) {
    struct stat st;
    if (stat(path, &st) == -1) return 0;
    return S_ISREG(st.st_mode);
}

// 忽略大小写的字符串匹配
char *strstr_nocase(const char *haystack, const char *needle) {
    if (*needle == '\0') return (char *)haystack;
    for (; *haystack != '\0'; haystack++) {
```

```

        if (tolower((unsigned char)*haystack) == tolower((unsigned char)*needle)) {
            const char *h = haystack, *n = needle;
            while (*h && *n && tolower((unsigned char)*h) == tolower((unsigned
char)*n)) {
                h++; n++;
            }
            if (*n == '\0') return (char *)haystack;
        }
    }
    return NULL;
}

```

// 关键词标红输出

```

void print_colored_line(const char *line, const char *keyword) {
    const char *pos = line;
    char *match;
    while ((match = strstr_nocase(pos, keyword)) != NULL) {
        fwrite(pos, 1, match - pos, stdout);
        printf("%s%s%s", COLOR_RED, keyword, COLOR_RESET);
        pos = match + strlen(keyword);
    }
    printf("%s\n", pos);
}

```

// 搜索单个文件

```

void search_single_file(const char *filename, const char *keyword) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        fprintf(stderr, " 无法打开文件: %s\n", filename);
        return;
    }

    char line[MAX_LINE_LEN];
    int line_num = 0, has_match = 0;
    while (fgets(line, MAX_LINE_LEN, fp) != NULL) {

```

```

        line_num++;
        line[strcspn(line, "\n")] = '\0';
        if (strstr_nocase(line, keyword) != NULL) {
            if (!has_match) {
                printf("=== 搜索结果 ===\n");
                has_match = 1;
            }
            printf("文件: %s, 行号: %d, 内容: ", filename, line_num);
            print_colored_line(line, keyword);
        }
    }
    fclose(fp);
    if (!has_match) printf("文件 【%s】 中未找到关键词! \n", filename);
}

// 搜索目录下所有文件
void search_dir_all_files(const char *dirpath, const char *keyword) {
    DIR *dir = opendir(dirpath);
    if (dir == NULL) {
        fprintf(stderr, "无法打开目录: %s\n", dirpath);
        return;
    }

    struct dirent *entry;
    char filepath[512];
    int total_match = 0;
    printf("正在搜索目录 【%s】 下的所有文件 ...\n", dirpath);

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
            continue;
        snprintf(filepath, sizeof(filepath), "%s/%s", dirpath, entry->d_name);
        if (is_file(filepath)) search_single_file(filepath, keyword);
    }
    closedir(dir);
}

```

```

        if (!total_match) printf("目录【%s】下未找到关键词！\n", dirpath);
    }

// 主函数
int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "使用方法: %s <文件 / 目录路径> <关键词>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

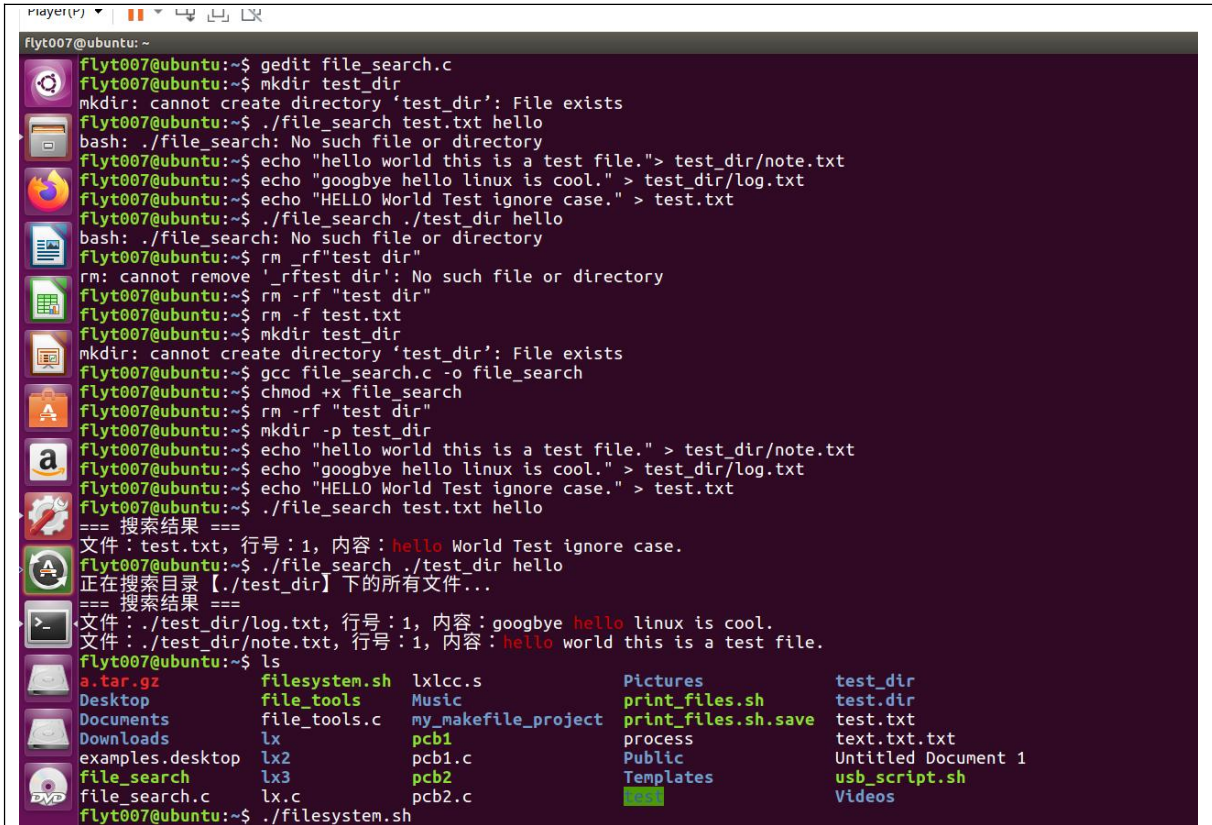
    char *target = argv[1], *keyword = argv[2];

    if (is_file(target)) search_single_file(target, keyword);
    else if (is_dir(target)) search_dir_all_files(target, keyword);
    else fprintf(stderr, "路径【%s】不存在或不是文件 / 目录！\n", target);

    return 0;
}

```

4、程序运行过程及结果:



```

flyt007@ubuntu: ~
flyt007@ubuntu:~$ gedit file_search.c
flyt007@ubuntu:~$ mkdir test_dir
mkdir: cannot create directory 'test_dir': File exists
flyt007@ubuntu:~$ ./file_search test.txt hello
bash: ./file_search: No such file or directory
flyt007@ubuntu:~$ echo "hello world this is a test file." > test_dir/note.txt
flyt007@ubuntu:~$ echo "googbye hello linux is cool." > test_dir/log.txt
flyt007@ubuntu:~$ echo "HELLO World Test ignore case." > test.txt
flyt007@ubuntu:~$ ./file_search ./test_dir hello
bash: ./file_search: No such file or directory
flyt007@ubuntu:~$ rm -rf "test_dir"
rm: cannot remove '_rf' test_dir': No such file or directory
flyt007@ubuntu:~$ rm -rf "test_dir"
flyt007@ubuntu:~$ rm -f test.txt
flyt007@ubuntu:~$ mkdir test_dir
mkdir: cannot create directory 'test_dir': File exists
flyt007@ubuntu:~$ gcc file_search.c -o file_search
flyt007@ubuntu:~$ chmod +x file_search
flyt007@ubuntu:~$ rm -rf "test_dir"
flyt007@ubuntu:~$ mkdir -p test_dir
flyt007@ubuntu:~$ echo "hello world this is a test file." > test_dir/note.txt
flyt007@ubuntu:~$ echo "googbye hello linux is cool." > test_dir/log.txt
flyt007@ubuntu:~$ echo "HELLO World Test ignore case." > test.txt
flyt007@ubuntu:~$ ./file_search test.txt hello
=== 搜索结果 ===
文件: test.txt, 行号: 1, 内容: hello World Test ignore case.
flyt007@ubuntu:~$ ./file_search ./test_dir hello
正在搜索目录【./test_dir】下的所有文件...
=== 搜索结果 ===
文件: ./test_dir/log.txt, 行号: 1, 内容: googbye hello linux is cool.
文件: ./test_dir/note.txt, 行号: 1, 内容: hello world this is a test file.
flyt007@ubuntu:~$ ls
a.tar.gz      filesystem.sh  lxlcc.s      Pictures      test_dir
Desktop       file_tools    Music         print_files.sh  test_dir
Documents     file_tools.c  my_makefile_project  print_files.sh.save  test.txt
Downloads     lx            pcb1          process        text.txt.txt
examples.desktop  lx2          pcb1.c       Public         Untitled Document 1
file_search   lx3            pcb2          Templates      usb_script.sh
file_search.c lx.c           pcb2.c       test           Videos
flyt007@ubuntu:~$ ./filesystem.sh

```

总结：

本次实践围绕Linux 系统核心技术，完成了 “磁盘管理” “文件操作” “进程 / 线程控制” “嵌入式应用开发” 四个模块的项目开发，覆盖了 Linux 系统编程、文件管理、进程通信、多线程同步等核心知识点，形成了从基础操作到综合应用的完整技术链。

四个项目以 “Linux 系统能力分层实践” 为核心逻辑，层层递进：

基础层（项目一：磁盘管理 + 项目二：文件操作）：聚焦 Linux 文件系统的基础操作，实现磁盘挂载 / 卸载、文件 / 目录的创建 / 修改 / 删除等功能，掌握系统命令调用与 Shell 脚本开发；

进阶层（项目三：进程 / 线程控制）：深入系统内核级通信与调度，实现基于信号的进程通信、多线程并发与同步，理解 Linux 的进程隔离与线程共享特性；

综合层（项目四：嵌入式 Linux 应用开发）：整合前序技术，开发轻量级嵌入式文件搜索工具，适配资源有限的嵌入式场景，实现从 “基础操作” 到 “实用工具” 的落地。

本次实践以 “从基础操作到综合应用” 为脉络，完成了 Linux 系统开发的全流程实践，既巩固了理论知识，也实现了技术的落地应用。后续可基于现有成果，扩展 “多进程并发搜索” “磁盘容量监控” 等功能，进一步提升工具的实用性，同时深入 Linux 内核机制，探索更底层的系统开发技术。

教 师 评 语

指 导 教 师 :

年 月 日